LAB Assignment No 10

K means Clustering Schemes in Machine Learning

Question 1

Write Python code to implement K-Means clustering from scratch using the following data points:

P1(1,3), P2(2,2), P3(5,8), P4(8,5), P5(3,9), P6(10,7), P7(3,3), P8(9,4), P9(3,7)

Tasks:

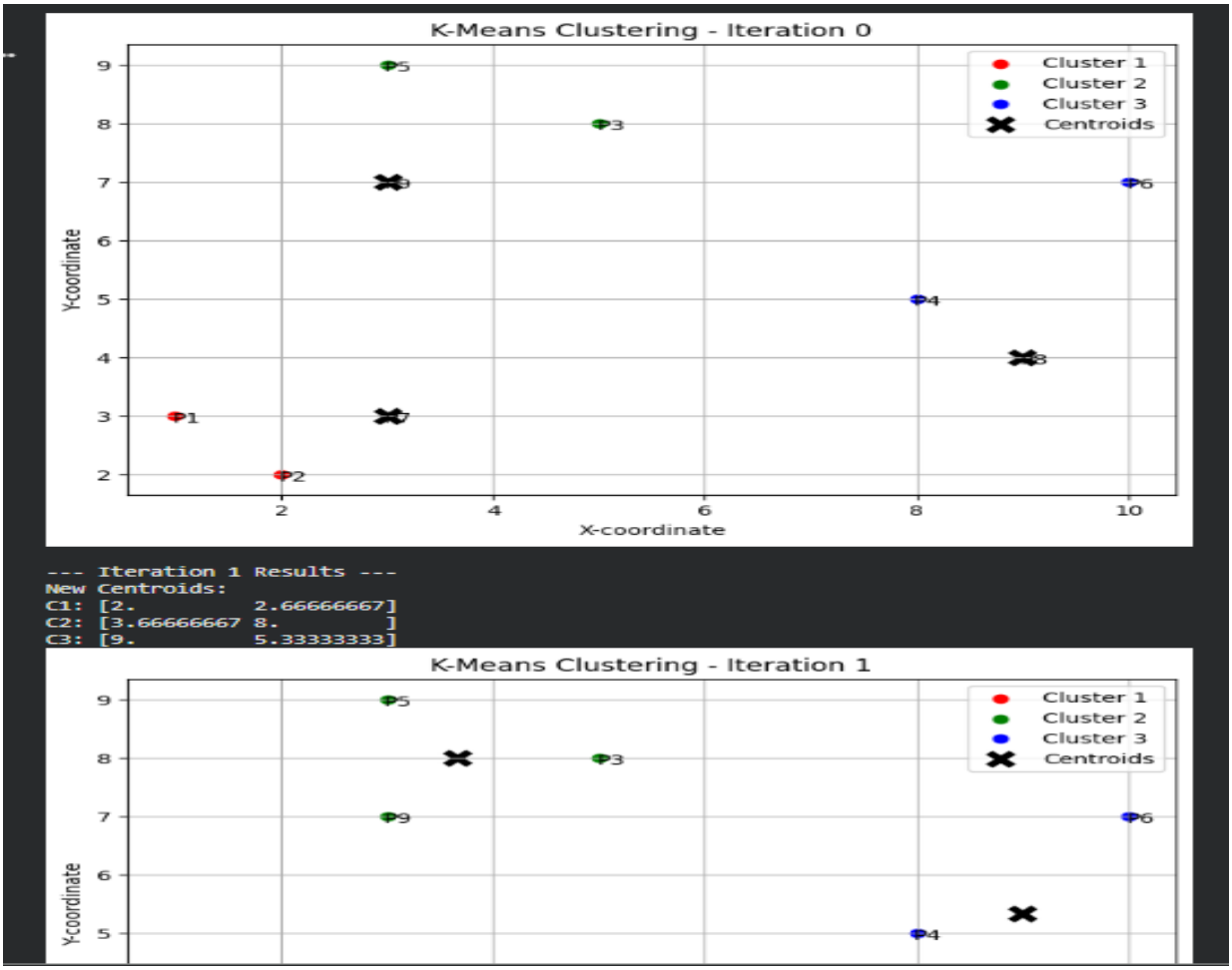
1. Use **K = 3** clusters
2. Initialize centroids as
   - C1 = P7(3,3)
   - C2 = P9(3,7)
   - C3 = P8(9,4)
3. Perform **2 iterations** manually in code
4. Plot all points and centroids
5. **Label all points (P1…P9)**
6. Sketch the clusters using matplot library in python

Code:

```
import numpy as np

import matplotlib.pyplot as plt

points = np.array([ [1, 3], [2, 2], [5, 8], [8, 5], [3, 9], [10, 7], [3, 3], [9, 4], [3, 7]])

point_labels = [f'P{i+1}' for i in range(len(points))]K = 3

centroids = np.array([

   points[6],   points[8],  points[7]  ])

def assign_clusters(data, centroids):
```

```
def euclidean_distance(a, b):
   """Computes the Euclidean distance between two points."""
   return np.sqrt(np.sum((a - b)**2))
assignments = []
   distances = []
   for point in data:
      dist_to_centroids = [euclidean_distance(point, c) for c in centroids]
      assignments.append(np.argmin(dist_to_centroids))
      distances.append(dist_to_centroids)
   return np.array(assignments),
 np.array(distances)
def update_centroids(data, assignments, K):
 new_centroids = []
   for k in range(K):
      cluster_points = data[assignments == k]
      if len(cluster_points) > 0:
         new_centroids.append(cluster_points.mean(axis=0))else:
         new_centroids.append(centroids[k])
   return np.array(new_centroids)
def plot_clusters(data, assignments, centroids, iteration):
   plt.figure(figsize=(8, 6))
   colors = ['r', 'g', 'b', 'y', 'c', 'm']
   for i in range(K):
 cluster_points = data[assignments == i]
```

```
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1],
                c=colors[i], label=f'Cluster {i+1}')
  plt.scatter(centroids[:, 0], centroids[:, 1],
              marker='X', s=200, c='k', label='Centroids', edgecolor='w')
  for i, txt in enumerate(point_labels):
    plt.annotate(txt, (data[i, 0], data[i, 1]), textcoords="offset points", xytext=(5,-5),
ha='center')
  plt.title(f'K-Means Clustering - Iteration {iteration}')
  plt.xlabel('X-coordinate')
  plt.ylabel('Y-coordinate')
  plt.legend()
  plt.grid(True)
  plt.show()
print("--- Initial State ---")
assignments_0, _ = assign_clusters(points, centroids)
plot_clusters(points, assignments_0, centroids, 0)
assignments_1, _ = assign_clusters(points, centroids)
centroids_1 = update_centroids(points, assignments_1, K)
print(f"\n--- Iteration 1 Results ---")
print(f"New Centroids:\nC1: {centroids_1[0]}\nC2: {centroids_1[1]}\nC3: {centroids_1[2]}")
centroids = centroids_1 # Update centroids for the next iteration
plot_clusters(points, assignments_1, centroids_1, 1)
assignments_2, _ = assign_clusters(points, centroids)
centroids_2 = update_centroids(points, assignments_2, K)
print(f"\n--- Iteration 2 Results ---")
print(f"New Centroids:\nC1: {centroids_2[0]}\nC2: {centroids_2[1]}\nC3: {centroids_2[2]}")
centroids = centroids_2 # Final centroids
plot_clusters(points, assignments_2, centroids_2, 2)
```



Question No. 2

Use the scikit-learn KMeans() library to cluster the same points.

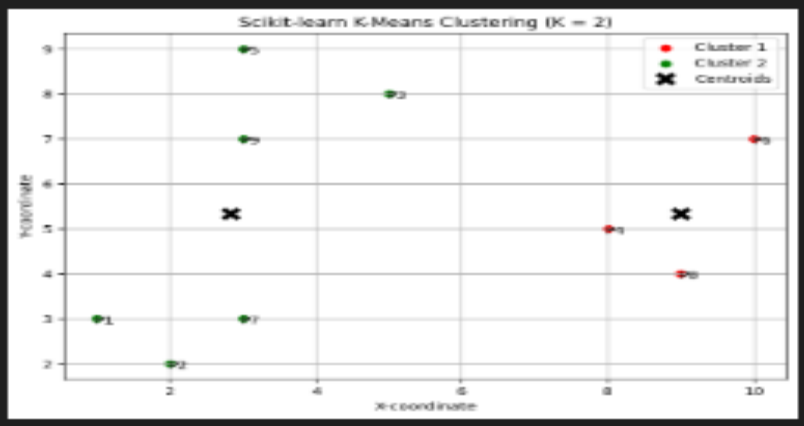
P1(1,3), P2(2,2), P3(5,8), P4(8,5), P5(3,9), P6(10,7), P7(3,3), P8(9,4), P9(3,7)

Tasks:

1. Use K = 2, 3, and 4
2. Plot the clustering result for each K
3. Compare:
    - Number of points in each cluster
    - Final centroid locations
4. Draw the 3 graphs in your lab copy and explain how the shapes change with K.

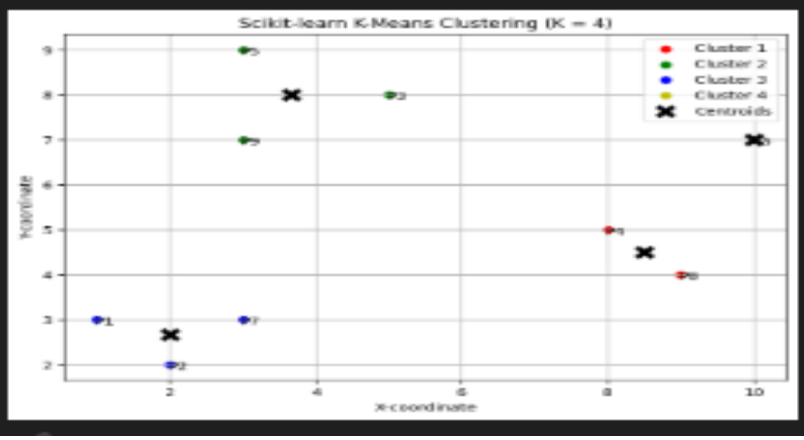
```
def run_sklearn_kmeans(data, k_val, point_labels):
    """Runs Scikit-learn KMeans for a given K and plots the result."""
    # n_init='auto' and random_state for reproducibility
    kmeans = KMeans(n_clusters=k_val, random_state=42, n_init='auto')
    kmeans.fit(data) labels = kmeans.labels_ centroids = kmeans.cluster_centers_
    cluster_counts = np.bincount(labels)
    print(f"\n--- Scikit-learn K-Means with K = {k_val} ---")
    print(f"Final Centroid Locations:\n{centroids}")
    print(f"Number of points in each cluster: {cluster_counts}")
    plt.figure(figsize=(8, 6)) colors = ['r', 'g', 'b', 'y']
    for i in range(k_val):
        cluster_points = data[labels == i]
        plt.scatter(cluster_points[:, 0], cluster_points[:, 1],c=colors[i], label=f'Cluster {i+1}')
    plt.scatter(centroids[:, 0], centroids[:, 1],
   marker='X', s=200, c='k', label='Centroids', edgecolor='w', zorder=10)
    for i, txt in enumerate(point_labels):
        plt.annotate(txt, (data[i, 0], data[i, 1]), textcoords="offset points", xytext=(5,-5),
ha='center')
    plt.title(f'Scikit-learn K-Means Clustering (K = {k_val})')
    plt.xlabel('X-coordinate')
    plt.ylabel('Y-coordinate')
    plt.legend()
    plt.grid(True)
    plt.show()
run_sklearn_kmeans(POINTS_9, 2, POINT_LABELS_9)
run_sklearn_kmeans(POINTS_9, 3, POINT_LABELS_9)
run_sklearn_kmeans(POINTS_9, 4, POINT_LABELS_9)
```

```
--- Scikit-learn K-Means with K = 2 ---
Final Centroid Locations:
[[9.         6.        ]
 [2.        6.        ]]
Number of points in each cluster: [4 6]
```




```
--- Scikit-learn K-Means with K = 3 ---
Final Centroid Locations:
[[9.         6.        ]
 [2.         8.        ]
 [2.         2.        ]]
Number of points in each cluster: [4 2 4]
```

```
--- Scikit-learn K-Means with K = 4 ---
Final Centroid Locations:
[[ 8.5        4.5       ]
 [ 2.         8.        ]
 [ 2.         2.        ]
 [10.         7.        ]]
Number of points in each cluster: [2 2 2 4]
```



Question 3 — Add a New User Point and Re-Cluster

Given the original 9 points, **add a new user**: **P10(6,2)**

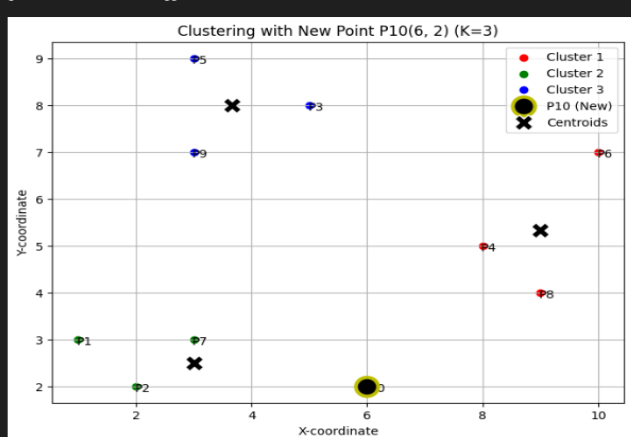
Tasks:

1. Run K-Means using K = 3
2. Plot the graph with all 10 points
3. Identify:
    - Which cluster P10 joins
    - How centroids shift after adding P10
4. Sketch before/after clusters in notebook
5. Write a short explanation about how a new data point affects clustering.

```
P10 = np.array([6, 2])

points_10 = np.vstack([POINTS_9, P10])

point_labels_10 = POINT_LABELS_9 + ['P10']

kmeans_q3 = KMeans(n_clusters=K_VALUE, random_state=42, n_init='auto')

kmeans_q3.fit(points_10)

labels_q3 = kmeans_q3.labels_

centroids_q3 = kmeans_q3.cluster_centers_

P10_cluster_index = labels_q3[-1]

print(f"--- K-Means with New Point P10(6, 2) ---")

print(f"P10(6, 2) joined Cluster: {P10_cluster_index + 1}")

print(f"Final Centroid Locations (10 points):\n{centroids_q3}")

plt.figure(figsize=(8, 6))

colors = ['r', 'g', 'b']

for i in range(K_VALUE):

   cluster_points = points_10[labels_q3 == i]

  plt.scatter(cluster_points[:, 0], cluster_points[:, 1], c=colors[i], label=f'Cluster {i+1}')
```

```
plt.scatter(P10[0], P10[1], marker='o', s=300, c='k', edgecolor='y', linewidth=3, label='P10 (New)', zorder=10)

plt.scatter(centroids_q3[:, 0], centroids_q3[:, 1],  marker='X', s=200, c='k', label='Centroids', edgecolor='w', zorder=10)

for i, txt in enumerate(point_labels_10):

    plt.annotate(txt, (points_10[i, 0], points_10[i, 1]), textcoords="offset points", xytext=(5,-5), ha='center')

plt.title('Clustering with New Point P10(6, 2) (K=3)')

plt.xlabel('X-coordinate')

plt.ylabel('Y-coordinate')

plt.legend()

plt.grid(True)

plt.show()

print("\nExplanation:")

print("A new data point affects clustering by altering the mean of its assigned cluster.")

print("P10(6, 2), being located in the lower-central region, pulls the closest centroid (likely Cluster 1) down and to the right, causing that cluster's boundary to shift.")
```

```
--- K-Means with New Point P10(6, 2) ---
P10(6, 2) joined Cluster: 2
Final Centroid Locations (10 points):
[[9.         5.33333333]
 [3.         2.5       ]
 [3.66666667 8.        ]]
```




```
Explanation:
A new data point affects clustering by altering the mean of its assigned cluster.
P10(6, 2), being located in the lower-central region, pulls the closest centroid (likely Cluster 1) down and to the right, causing that cluster's boundary to shift.
```

Question 4 — Distance Table + First Iteration Manually

Using the given 9 points and initial centroids:

C1(3,3), C2(3,7), C3(9,4)

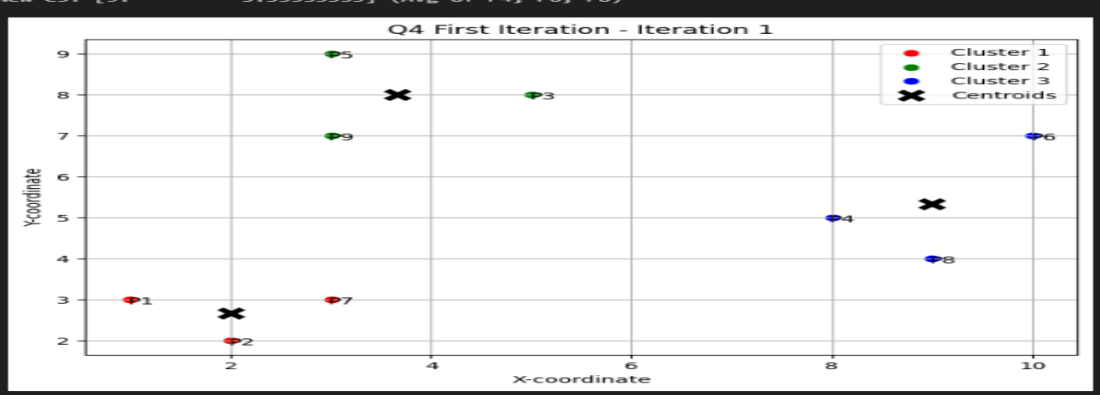
Tasks:

1. Compute **Euclidean distance** of each point to each centroid (manually or in python)
2. Create a distance table:

Point Dist to C1 Dist to C2 Dist to C3 Assigned Cluster

3. Perform **only the first iteration**
4. Compute **new centroids**
5. Plot the **first-iteration graph**
6. Draw the graph in your copy and show all labels.

```
C1_Q4 = np.array([3, 3])

C2_Q4 = np.array([3, 7])

C3_Q4 = np.array([9, 4])

centroids_Q4 = np.array([C1_Q4, C2_Q4, C3_Q4])

distances_Q4 = []

assignments_Q4 = []

for i, point in enumerate(POINTS_9):

    dist_c1 = euclidean_distance(point, C1_Q4)

    dist_c2 = euclidean_distance(point, C2_Q4)

    dist_c3 = euclidean_distance(point, C3_Q4)

    dists = [dist_c1, dist_c2, dist_c3]

    assigned_cluster_index = np.argmin(dists)

    distances_Q4.append(dists)

    assignments_Q4.append(assigned_cluster_index)
```

```
df_distances = pd.DataFrame(distances_Q4, columns=['Dist to C1', 'Dist to C2', 'Dist to C3'])

df_distances['Point'] = POINT_LABELS_9

df_distances['Assigned Cluster'] = [f'C{i+1}' for i in assignments_Q4]

df_distances = df_distances[['Point', 'Dist to C1', 'Dist to C2', 'Dist to C3', 'Assigned Cluster']]

def highlight_min(s):

    is_min = s == s.min()

    return ['background-color: yellow' if v else '' for v in is_min]

print("--- Distance Table (First Assignment) ---")

print(df_distances.to_string(index=False, float_format="%.2f"))

assignments_Q4 = np.array(assignments_Q4)

new_centroids_Q4 = update_centroids(POINTS_9, assignments_Q4, K_VALUE)

print("\n--- New Centroids (After First Iteration) ---")

print(f"New C1: {new_centroids_Q4[0]} (Avg of P1, P2, P7)")

print(f"New C2: {new_centroids_Q4[1]} (Avg of P3, P5, P9)")

print(f"New C3: {new_centroids_Q4[2]} (Avg of P4, P6, P8)")

plot_clusters(POINTS_9, assignments_Q4, new_centroids_Q4, 1, title_prefix="Q4 First Iteration")
```

```
--- Distance Table (First Assignment) ---
Point  Dist to C1  Dist to C2  Dist to C3 Assigned Cluster
   P1        2.00        4.47        8.06                C1
   P2        1.41        5.10        7.28                C1
   P3        5.39        2.24        5.66                C2
   P4        5.39        5.39        1.41                C3
   P5        6.00        2.00        7.81                C2
   P6        8.06        7.00        3.16                C3
   P7        0.00        4.00        6.08                C1
   P8        6.08        6.71        0.00                C3
   P9        4.00        0.00        6.71                C2

--- New Centroids (After First Iteration) ---
New C1: [2.         2.66666667] (Avg of P1, P2, P7)
New C2: [3.66666667 8.        ] (Avg of P3, P5, P9)
New C3: [9.         5.33333333] (Avg of P4, P6, P8)
```